

REINFORCEMENT LEARNING ALGORITHM RESEARCH AND ANALYSIS REPORT

Group5: Zeng XinYi, Liu FanZhe, Yuan KeFan

ABSTRACT

This report analyzes **Q-learning, DQN, Double DQN, Dueling DQN**, and **Prioritized Experience Replay (PER)** algorithms in reinforcement learning. The algorithm used in this group to evaluate performance is **Dueling DQN**, and the analysis of the parameters is reflected in **the second section** of the report.

1 Q-LEARNING

1.1 SINGLE PARAMETER ANALYSIS

The team first changed only a single parameter based on the original parameters to analyze the impact of the parameter on training. The original parameters were the following:

Learning rate	Discount factor	Epsilon decay	Epsilon min	Epsilon max
0.05	0.99	0.9999	0.1	1

Table 1: Original parameter

1.1.1 LEARNING RATE

The learning rate determines the extent to which the newly acquired information affects the original Q value. A smaller learning rate can avoid making too big a policy change, but the convergence speed is slow and the training time is long; a larger learning rate can quickly adjust the Q value in the right direction, but there may be an overfitting problem. The optimal value of the learning rate based on the original parameters is **0.05**.

1.1.2 DISCOUNT FACTOR

The discount factor is used to measure the impact of future rewards on the value of the current state. The optimal value of the discount factor based on the original parameters is **0.6**, which obtains a higher average reward. When the extreme value is not taken, the discount factor has little effect on the number of steps after convergence. The possible reason is that the environment is simple and the map size is small, the reward distribution is dense, and long-term planning may not be as effective as short-term decision making. If the discount factor is large, the agent may spend too much energy planning long-term paths and ignore the rewards that can be obtained quickly, resulting in a decrease in overall rewards.

In addition, we found that the size of the learning rate also affects the effect of the discount factor. If the learning rate is large, a high discount factor may make the Q value update too aggressive, leading to error propagation and instability; while lowering the discount factor can focus on the current state faster.

1.1.3 EPSILON RELATED PARAMETERS

"Epsilon min" defines the lower limit of the random exploration of the agent when selecting actions. Generally speaking, the smaller the Epsilon min, the better the reward value after the final convergence. We take **0.01** as the optimal value. The selection of this value has little effect on the number of steps after convergence.



Figure 1: discount factor (Q-learning)

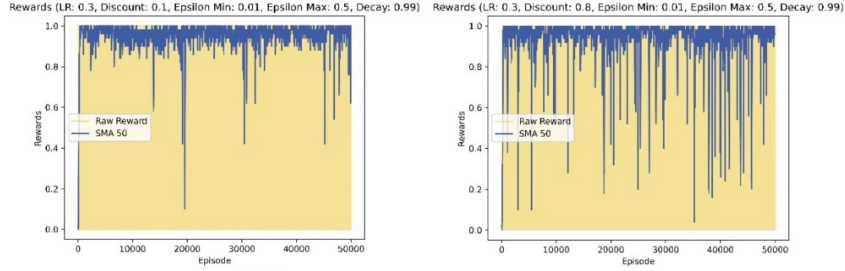


Figure 2: discount factor and learning rate (Q-learning)

In simple environments, a **smaller** epsilon min helps the agent use the known optimal policy most of the time, thereby optimizing reward and step performance. However, in complex environments, a larger epsilon min may be needed to avoid falling into local optimality.

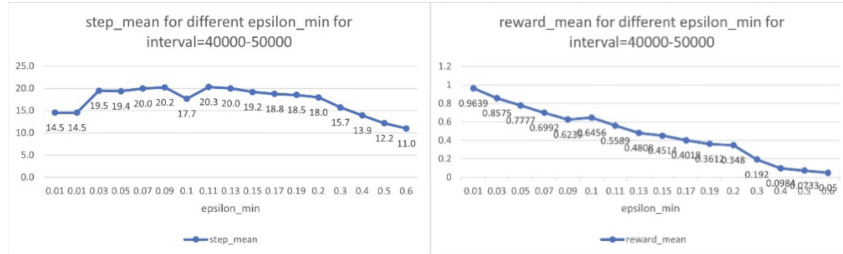


Figure 3: epsilon min (Q-learning)

“**Epsilon max**” defines the maximum probability of random exploration by the agent when selecting actions. A larger Epsilon Max helps the agent understand the environment more comprehensively, so when epsilon max is **1**, the model training performance is better than other parameter values. In the initial stage of training, high-probability random selection will take longer to converge to a better strategy. However, due to the simple environment and the sufficient number of training steps, in general, as long as the selection of epsilon max is not too small, it will not have much effect on the speed and results of training.

Because a smaller ‘**epsilon decay**’ means that the exploration rate drops too quickly, the agent can excessively rely on the initial exploration experience in the early stage, causing the learning process to converge prematurely to a suboptimal solution, so only the situation above 0.9 is explored. It is found that the higher the epsilon decay, the better the model effect after convergence, that is, the higher the average return value after convergence and the lower the average number of steps.

Although a higher ϵ decay rate will reduce training efficiency, in simple environments, due to the fast training characteristics of Q-learning, the speed can be ignored and the focus can be placed on the model convergence effect. Therefore, a higher decay rate ϵ can be used to enhance exploration.



Figure 4: epsilon max (Q-learning)

However, in complex environments or slow algorithms, the decay rate ϵ must be adjusted to balance exploration and utilization and improve training efficiency.

1.2 OPTIMAL PARAMETER COMBINATION

We take the **Cartesian product** of the parameters and explore the training results of different combinations of parameters, and find the set of parameters with the highest average reward after convergence:

Learning rate	Discount factor	Epsilon decay	Epsilon min	Epsilon max
0.06	0.6	0.99	0.01	0.5

Table 2: Optimal parameter (Q-learning)

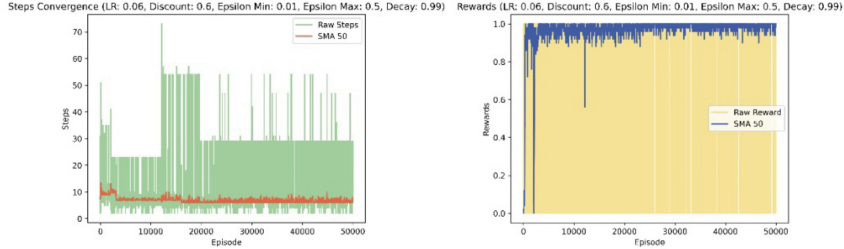


Figure 5: Optimal parameter combination (Q-learning)

2 COMPARISON AND ANALYSIS OF DQN, DOUBLE DQN, AND DUELING DQN

2.1 DOUBLE DQN ALGORITHM PRINCIPLE

Double DQN optimizes the estimation of the Q value during training.

DQN directly uses the target network to select and evaluate the Q value of the action (formula: $y = r + \gamma \max_{a'} Q'(s', a')$), but because the Q function is updated by maximizing the operation, it is easy to overestimate the Q value during learning.

In comparison, double DQN separates action selection and Q-value evaluation, uses the current evaluation network to select actions, and then uses the target network to calculate the target Q-value of this action. This reduces the risk of overestimating the Q-value and can estimate the Q-value more accurately (formula: $y = r + \gamma Q'(s', \arg \max_{a'} Q(s', a'))$).

In the code implementation, the main network self.model is first used to select the action corresponding to the next state (obtain the action index with the largest Q value in each next state), and

then the target network "self.target_network" is used to evaluate the Q value corresponding to the above selected action and calculate the target Q value.

2.2 DUELING DQN ALGORITHM PRINCIPLE

The last layer of DQN is a fully connected layer, which directly outputs n Q values. Dueling DQN obtains two indirect variables V (state value) and A (action advantage) through training, and then represents the Q value by their sum. V represents the average expectation of the Q value in the current state s (considering all optional actions). A represents how much the Q value exceeds the expected value when action a is selected.

2.3 HYPERPARAMETER COMPARISON AND ANALYSIS OF DQN, DOUBLE DQN, AND DUELING DQN

Since the algorithm used for evaluation is Dueling DQN, this section will focus more on the analysis of Dueling DQN.

Learning rate	Discount factor	Epsilon decay	Epsilon min
0.0006	0.93	0.999	0.005
Epsilon max	Memory capacity	Update Frequency	Clip grad norm
0.999	4000	10	3

Table 3: Hyperparameter selection for evaluating performance (Dueling DQN)

2.3.1 CLIP GRAD NORM

Since DQN is more susceptible to the influence of Q-value overestimation, which leads to large gradient changes, a **relatively small** gradient clipping norm threshold may need to be set in DQN to avoid gradient explosion. Double DQN and dueling DQN have higher tolerance for larger values, and the values that can be set will be **higher**.

When Dueling DQN updates the shared layer parameters, the gradient changes are more complicated because it affects the estimation of value and advantage at the same time. This complex gradient change may make the gradient amplitude more likely to be larger, so a higher gradient clipping norm threshold is required to constrain it.

As can be seen from the figure, when the gradient clipping norm threshold in the Dueling DQN algorithm is very small, the cumulative reward fluctuates greatly and the convergence speed is slow. When the gradient clipping norm threshold is greater than or equal to 10, the convergence speed is basically the same. The value with the largest average value of the cumulative reward and the smallest variance is selected (**the subsequent hyperparameter value selection is selected based on this reason if it is not specified**), and the gradient clipping norm threshold is **50**.

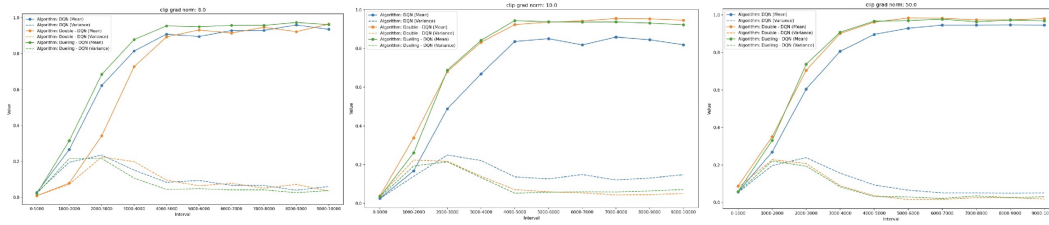


Figure 6: Comparison for clip grad norm

2.3.2 LEARNING RATE

A learning rate that is too large will lead to poor model stability and inability to converge; a learning rate that is too small will slow down the training speed. After training, we selected that the learning rate of DQN, double DQN and dueling DQN is **0.0006**, and plotted the mean and variance of the

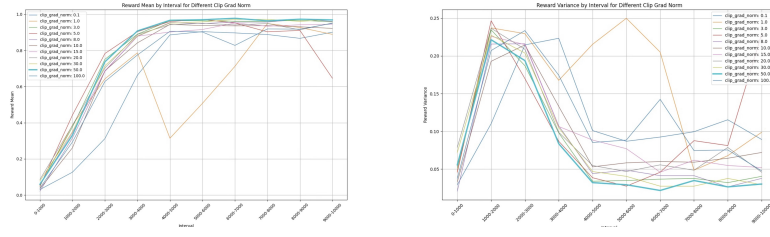


Figure 7: clip grad norm (dueling DQN)

two algorithms. It is found that DDQN and dueling converge faster, and the variance is smaller after convergence, which is more stable.

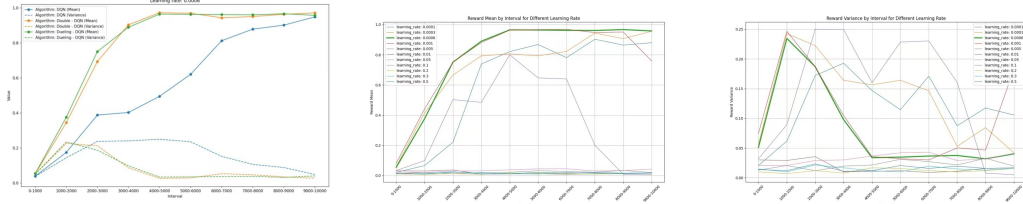


Figure 8: learning rate (comparison and dueling DQN)

2.3.3 DISCOUNT FACTOR

In DQN, in order to avoid suboptimal strategies caused by overestimating Q-values and over-emphasizing future rewards, a relatively small discount factor such as **0.9** is set. Double DQN and Dueling DQN set a discount factor of **0.93** that is closer to 1 because it can better handle the estimation of long-term rewards and more accurately balance the relationship between current rewards and future rewards.

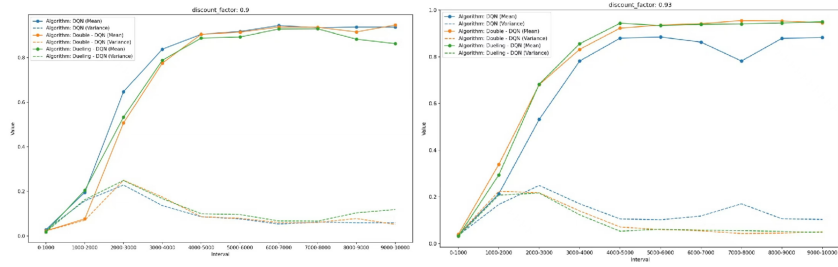


Figure 9: Comparison for discount factor

2.3.4 MEMORY CAPACITY

DDQN separates action selection and target Q-value calculation, allowing it to use samples more stably during learning, and has relatively low requirements on the timeliness and relevance of samples. It can use more historical samples for learning, because its learning process is less affected by inaccurate Q values in old samples. Therefore, DDQN can support a larger memory capacity, dueling DQN is more accurate for Q-values, so the value is higher.

2.3.5 EPSILON RELATED PARAMETERS

The optimal epsilon max value of the three algorithms is **0.999**. At this time, the reward mean is higher, the reward variance is lower, and the loss variance is lower and on a downward trend.

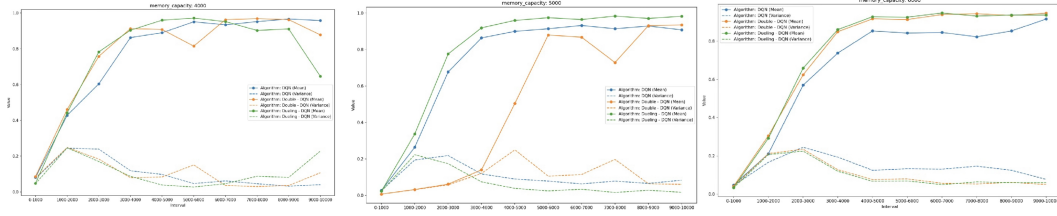
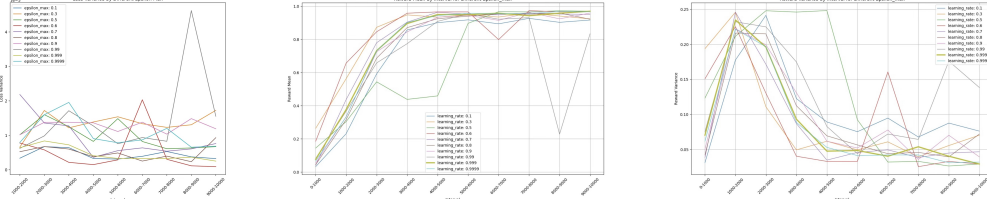


Figure 10: Comparison for memory capacity

Figure 11: ϵ max (dueling DQN)

Similarly, we conducted the same analysis for epsilon min and epsilon decay. The accuracy and stability of DDQN and dueling DQN for Q-value estimation increase in turn, so they can withstand higher epsilon min and epsilon decay.

2.3.6 UPDATE FREQUENCY

DQN and Double DQN estimate Q values based on traditional multi-layer fully connected networks. This simple and direct network structure is relatively easy to maintain stability during the update process. Therefore, when updating the main network and the target network, the update frequency can be appropriately increased to **30**. The network structure of dueling DQN is relatively complex, so **15** is selected.

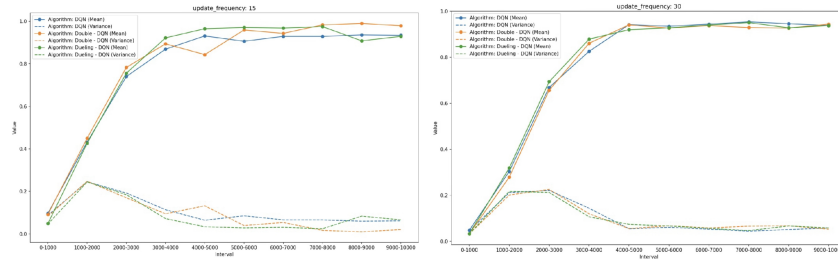


Figure 12: Comparison for update frequency

2.4 ALGORITHM SUMMARY

Double DQN improves learning stability and convergence speed, while Dueling dqn reduces unnecessary parameter learning and achieves more efficient estimation and policy optimization, all because the two algorithms **reduce the problem of overestimation of Q value**. However, both algorithms increase computational complexity and sensitivity to hyperparameter adjustment.

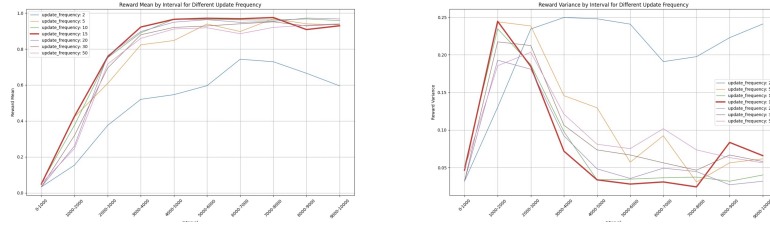


Figure 13: update frequency (dueling DQN)

3 PRIORITIZED EXPERIENCE REPLAY (PER) ALGORITHM

The PER algorithm performs non-uniform sampling based on the importance (priority) of experience, making it more likely that the agent will learn experiences that are more valuable for training, thereby improving learning efficiency and performance.

In the specific implementation of the code, the priority of experience is determined based on the absolute value of the TD error. The larger the error, the higher the priority of the experience. And it is stored and sampled through the SumTree data structure. The importance sampling weight (IS weight) is calculated to correct the bias of non-uniform sampling, and the priority of experience is updated after each learning.

3.1 HYPERPARAMETER ANALYSIS OF PER

3.1.1 PER ALPHA

“Per alpha” is used to control the calculation method of experience priority. By adjusting “PER0 alpha”, the priority experience playback can pay more attention to the experience with high priority, thereby improving learning efficiency, but it may also cause some problems such as over-focusing on certain experiences.

When it is smaller than 0.2, it converges early, has high benefits, and is more stable in learning. It is not easily misled by abnormally high-priority experiences, but it falls into local optimality. When it is 0.8, it converges early and the learning process focuses on key experiences more quickly, but it may also overfit these experiences. When it is 0.4, the optimal strategy is found in the learning process, and a balance is found in the influence of experience priority in the sampling process.

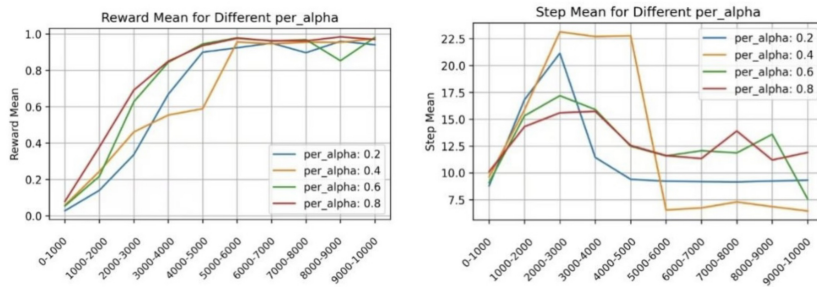


Figure 14: PER alpha

3.1.2 PER BETA

“Per beta” is used to correct the bias caused by non-uniform sampling. If the β value is too high, learning may focus only on some samples with high experience values, resulting in unstable learning or even finding only the best local solution. A smaller β value can make the algorithm more diverse and reduce the risk of overfitting a certain strategy, but it may slow down learning. In the actual

training process of the experimental environment, the best effect was achieved when PER beta was as small as 0.2.

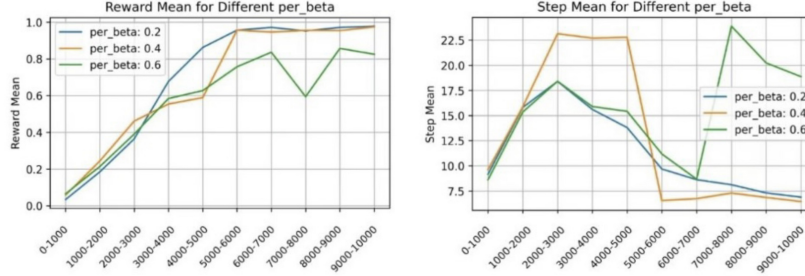


Figure 15: PER beta

3.2 DISADVANTAGES OF THE PER ALGORITHM

In the actual training process of the project experiment, due to the relatively simple experimental environment, the effect of adding PER to the Double DQN algorithm is not as good as that of the pure Double DQN. The following is the reason analysis:

- The PER algorithm accelerates the learning process by prioritizing the sampling of experiences with higher TD errors, which is particularly effective in complex environments because some states in these environments may be more critical to the learning of strategies. However, the advantage is not obvious in simple environments.
- In simple environments, the agent may be able to learn a better strategy quickly, so there is no need to update the Q value frequently. Therefore, the advantage of the PER algorithm in accelerating the learning process through priority sampling is not obvious.
- In terms of training speed, the related priority operations in the PER algorithm increase the complexity of the algorithm, requiring additional computing resources and memory to store relevant information, which may not be worth the loss in simple environments or resource-constrained scenarios.

So in actual training, it is necessary to choose an algorithm based on the actual situation such as the size of the environment and computing resources, rather than blindly using more advanced algorithms, which may reduce the training efficiency and effect.

4 CONCLUSION

For the Q-learning algorithm, in the simple environment of this experiment, a small learning rate, a moderate discount factor and a reasonable epsilon parameter setting can help improve the training effect. The optimal parameter combination determined performs well in terms of convergence speed and reward acquisition, but different environments may require re-adjustment of parameters.

For the DQN series of algorithms, Double DQN effectively reduces the risk of Q value overestimation by separating action selection and Q value evaluation, and improves learning stability and convergence speed. Dueling DQN uses a unique Q value representation method to reduce unnecessary parameter learning, achieve efficient estimation and strategy optimization, and the complex network structure makes it sensitive to hyperparameter adjustment.

For the PER algorithm, the learning efficiency is improved by prioritizing the sampling of key experiences, but the advantage is not obvious in a simple environment, and the learning effect may even be affected by improper priority setting. In practical applications, factors such as environmental complexity and computing resources should be considered comprehensively to decide whether to use the PER algorithm and how to adjust the parameters to avoid blindly using advanced algorithms to reduce efficiency.